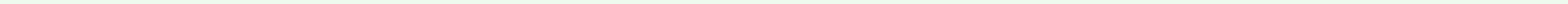


VISION TRANSFORMERS

INSTRUCTOR : Dr. BINOD KUMAR

INDUSRI SIRAMSETTY (B22AI039)



The Problem



The Challenge: Real-Time Security Threats

The Problem:

- Unauthorized drones pose significant security and privacy risks to critical infrastructure (airports, military bases, public events).
- Detection must be fast, accurate, and autonomous.

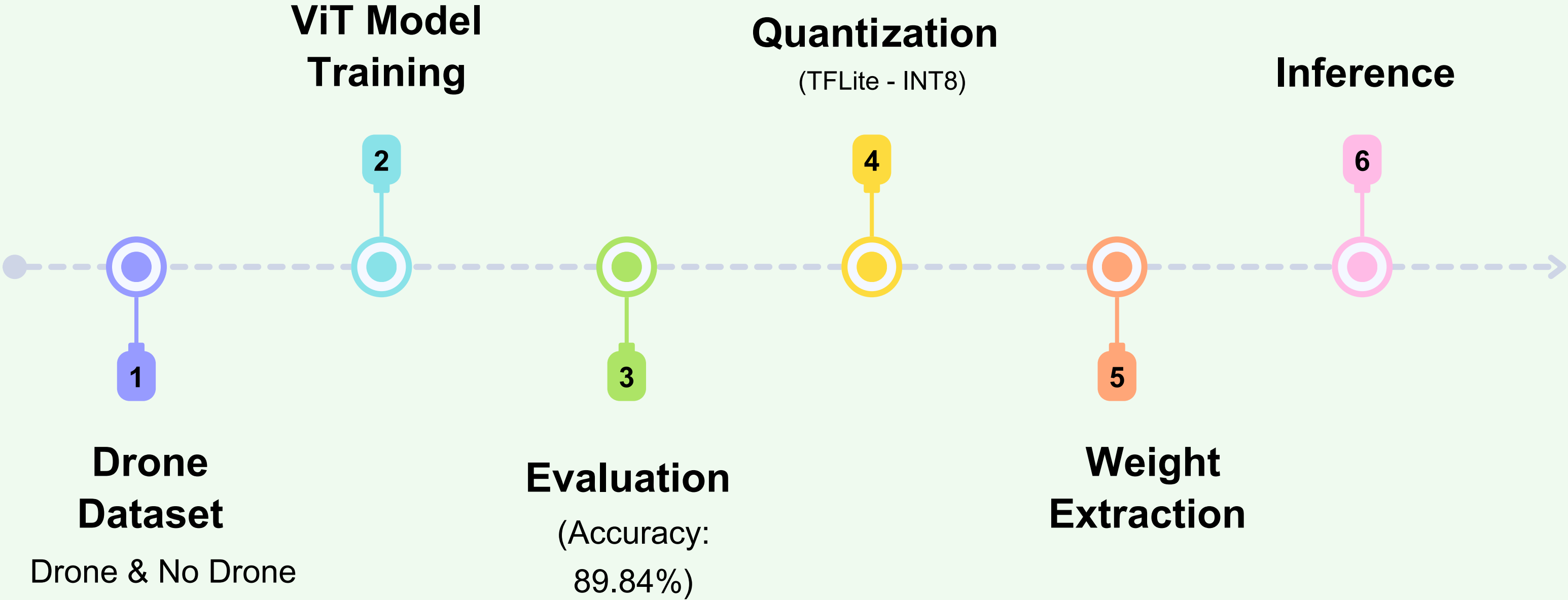
The Gap:

- High-accuracy models like Vision Transformers (ViT) are too large and slow for low-power edge devices (like FPGAs, microcontrollers, or on-drone hardware).
- You can't just run a 400 MB model on a tiny security camera.

The Objective:

- To **Design, Train, Quantize and inference** a ViT model that is:
 1. **Accurate** (High detection rate)
 2. **Lightweight** (Small memory footprint)
 3. **Fast** (Low-latency inference)
 4. **inference** (Ready for hardware implementation)

WORK FLOW



Model Development (ViT)

Building the Baseline ViT Model

Why a Vision Transformer?

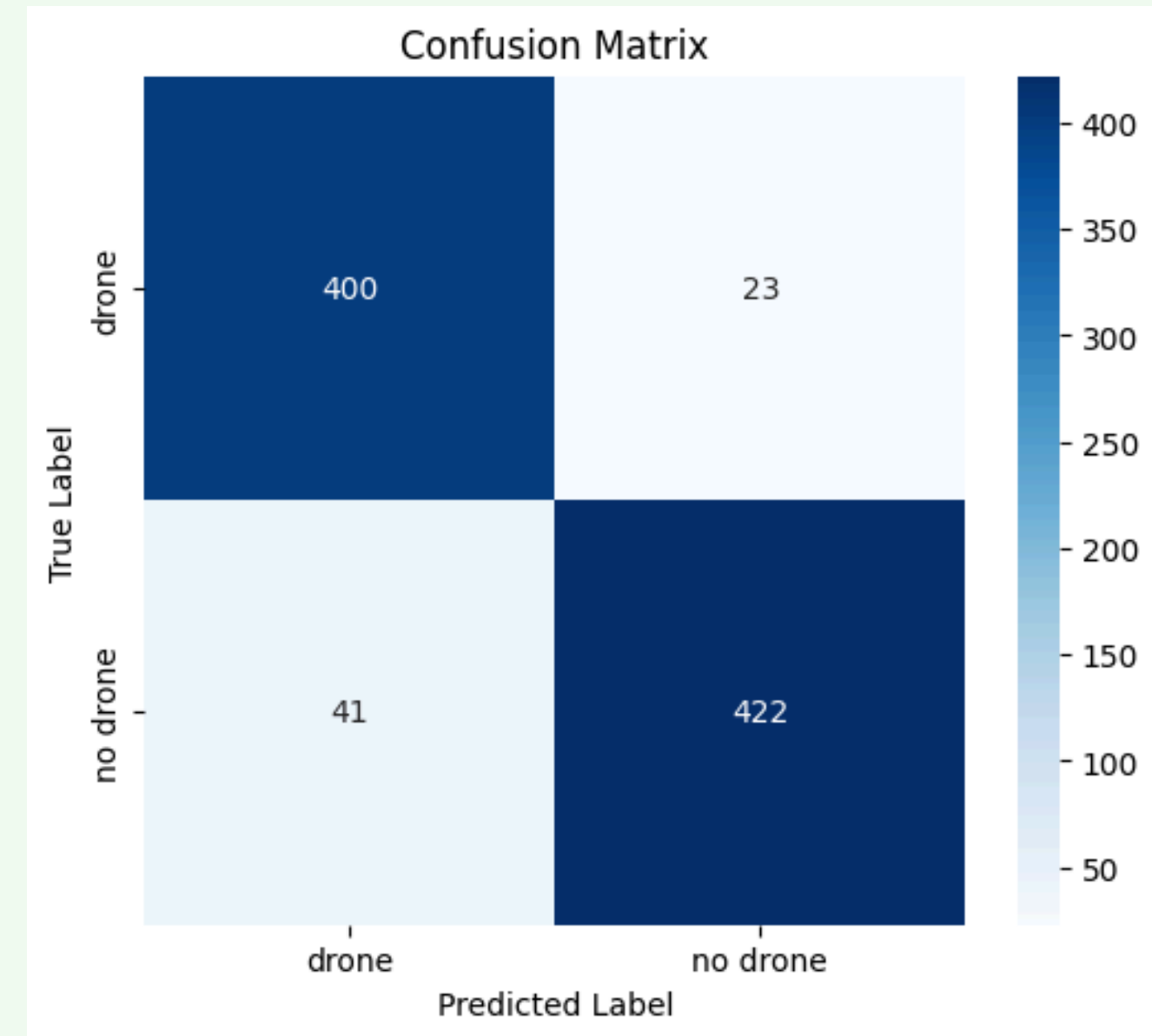
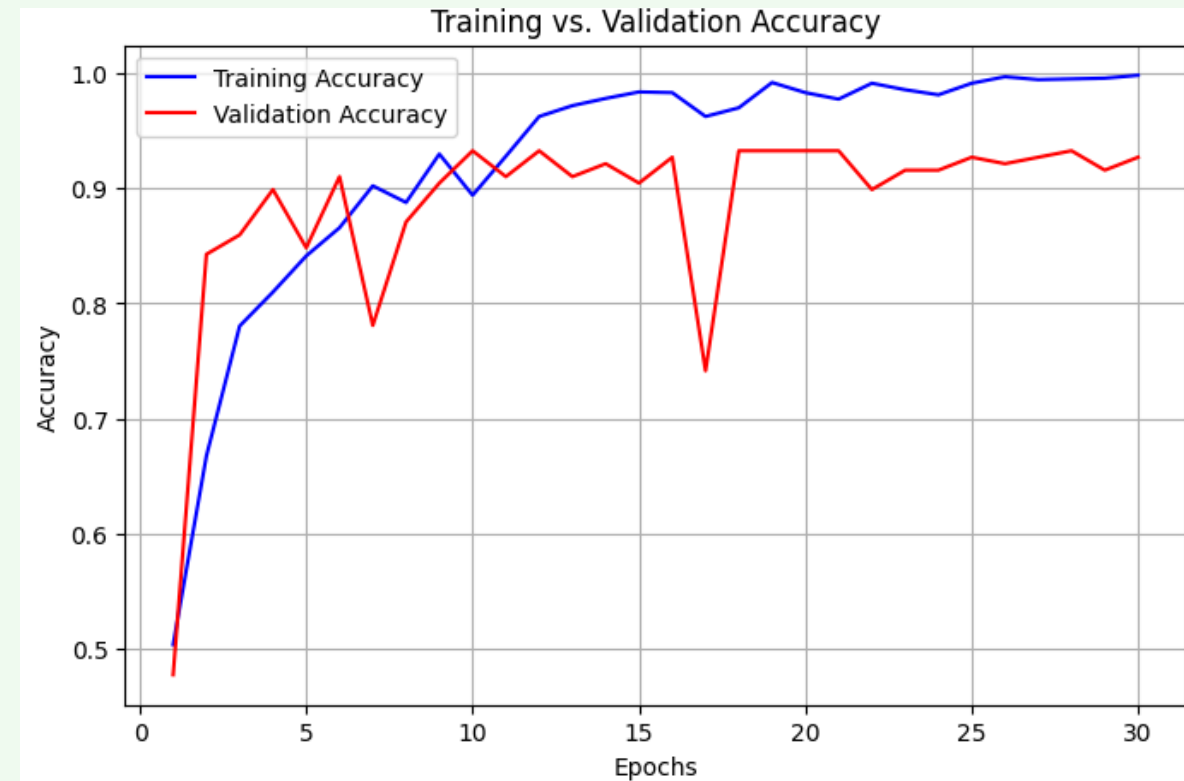
- Moves beyond standard CNNs by using Multi-Head Attention (MHA), allowing the model to learn complex relationships between different parts of an image.

Key Architecture

Parameters:

- Input Image: 299 x 299 (Resized to 128x128 for ViT Core)
- Patching: Image split into 8 x 8
- patchesEmbedding: Patches are flattened and projected to a 64-dimension space, combined with positional embedding.
- Transformer Block: 1 Layer with 2 Attention Heads Classes: 2 ("drone", "no drone")

Baseline Model Performance (FP32)



- Final Test Accuracy: 89.84%
- Classification Report (Key Insight):
Drone Recall: 0.93
No Drone Recall: 0.87
- (Put your Keras Confusion Matrix plot here: `[[209, 16], [29, 189]]`)

The Deployment Problem

- Our Keras model achieved high accuracy, but it has a critical flaw for real-world use...

Baseline Keras Model Size (FP32)

408.86 MB

This is **far too large** for any embedded system, mobile app, or edge device.

- This is un-deployable on any resource constrained edge device.
- We need a model that is not just accurate, but also small and fast.

Model Optimization

Solution: Post-Training Quantization(PTQ)

- We used TensorFlow Lite (TFLite) to convert our model. Quantization is the process of reducing the precision of the model's weights.
- A technique to convert the model's 32-bit floating-point weights (like 3.14159) into 8-bit integers (like 1-255).

The Goal:

- Drastically reduce model size and leverage integer-only math for massive speedups on compatible hardware.

QUANTIZATION RESULTS

Metrics	Original Keras	Quantized Model	Results
• MODEL SIZE	• 408.86MB	• 34.17 MB	• 12.0x smaller
• ACCURACCY	• 89.84%	• 87.81%	• ~2% NEGLIGIBLE DROP

Test accuracy of quantized model: **87.81%**

Confusion Matrix (Quantized):

[[202 23]
[31 187]]

```
[ 31 187]]
Classification Report (Quantized):
              precision    recall  f1-score   support

   drone         0.87         0.90         0.88         225
  no drone         0.89         0.86         0.87         218

   accuracy              0.88         443
  macro avg              0.88         443
 weighted avg              0.88         443
```

INFERENCE

- Used TFLite Interpreter to inspect the quantized model and find the tensor indices for all weights.
- Extracted all weights (MHA, MLP, Layer Norm, etc.) and saved them to .txt files.
- Wrote pure NumPy functions to simulate each Keras layer: `patch()`, `lp()` (Linear Projection), `mha()`, `mlp()`, `gelu()`, `softmax()`.
- Loads the `.txt` weights and executes each function in the correct order

- **OUTPUT:**

Quantized:

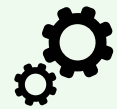
Original:

```
dense2  [-11.54709614 -14.27600517]
[0.9387111  0.0612889]
```

```
dense2  [-11.53631215 -14.74945451]
[0.96132586  0.03867414]
```

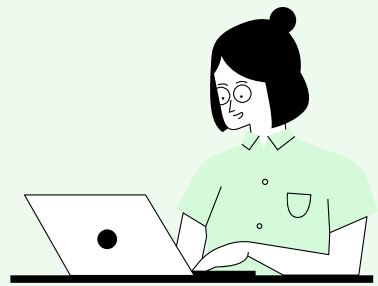
DeiT

Project Objectives: From Training to Deployment

-  Fine-Tune: Train a high-accuracy DeiT (Data-efficient Image Transformer) model on our custom drone dataset.
-  Optimize: Apply model quantization to drastically reduce model size and inference latency for on-device deployment.
-  Explore: Investigate multiple quantization pipelines, including TFLite, PyTorch, and advanced sub-8-bit simulations.
-  Analyze: Compare the FP32 vs. INT8 models on three key metrics: Accuracy, Model Size, and Inference Speed.

Methodology

- We used the Hugging Face transformers library to fine-tune the model on our drone dataset.



Final Accuracy:

94.77%

(Compared to 89.84% from our custom ViT)

- Training a 94.77% accurate model is good, but is it efficient? We applied Tflite Dynamic Quantization.

DeiT Quantization (PyTorch)

DeiT Model: Original vs. Quantize

Metric	Original DeiT Model (FP32)	Quantized DeiT Model (INT8)
Model Size	1721.46 MB (Full Checkpoint)	84.44 MB (Weights Only)
Inference Time	1973.53 ms	1584.84 ms
Prediction	"drone"	"drone"

Experiment: TFLite & Custom Bits

- To explore deployment on the most constrained hardware, we built pipeline to simulate extreme quantization (4-bit & 6-bit).

4-bit & 6-bit Precision Simulation

We simulated ultra-low precision by "trimming" the extracted INT8 weights. This proves a methodology for targeting FPGAs or custom hardware.

- Process: Loaded INT8 weights (range [-128, 127]) from .txt files.
- 6-bit Simulation: Clamped weights to [-32, 31].
- 4-bit Simulation: Clamped weights to [-8, 7].
- Result: Successfully generated 336 new custom bit-width files, validating the compression technique

TFLite & Custom Bit-Width Results

Model / Precision	Precision (Bits)	Relative Model Size (Theoretical)
Keras Baseline	Float32	~100%
TFLite (FIQ)	INT8	~25% (4x Reduction)
Simulated (6 bit)	INT6	~18.75% (5.3x Reduction)
Simulated (4 bit)	INT4	~12.5% (8x Reduction)

Novelty in ViT

The Core Challenge

- Standard ViT classifiers are "closed-set." They can only classify the 5 classes they were trained on (car, van, etc.).
- If shown a 'truck' or 'motorcycle', they will fail and misclassify it as one of the 5 known classes. They cannot say "I don't know."

Goal:

- The goal was to add novelty by integrating concepts from the **NeurIPS paper** (on in-context 1-NN) to create an "open-set" ViT-kNN system that can:
- Correctly classify the 5 "known" classes.
- If an "**unknown**" image is given, identify it as "Unknown Class".
- show the "relationship" by providing the closeness percentage to its closest matching known class.

Implementation: From Training to Calibration

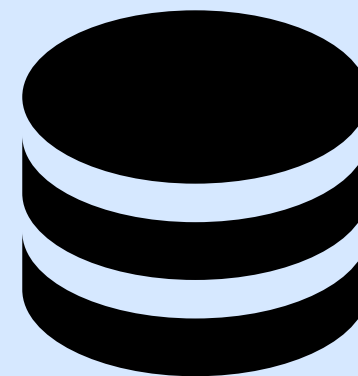
1. Train ViT

- Trained the 2-Head, 2-Layer ViT model on the 5-class VisDrone dataset.



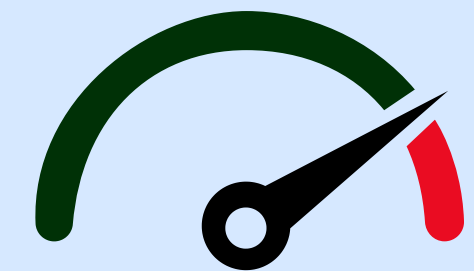
2. Build Database

- Created a "feature database" by passing all 17,510 training images through the ViT.
- Used L2-Normalization and Cosine Distance for robust matching.



3. Calibrate Threshold

- Used the 1,946 validation images to find the optimal "Unknown" threshold.
- A 99th percentile cutoff (0.0174) was chosen for a balanced system.



Technical Challenge & Solution



Problem:

`'ERROR_NEEDS_FLEX_OPS'`



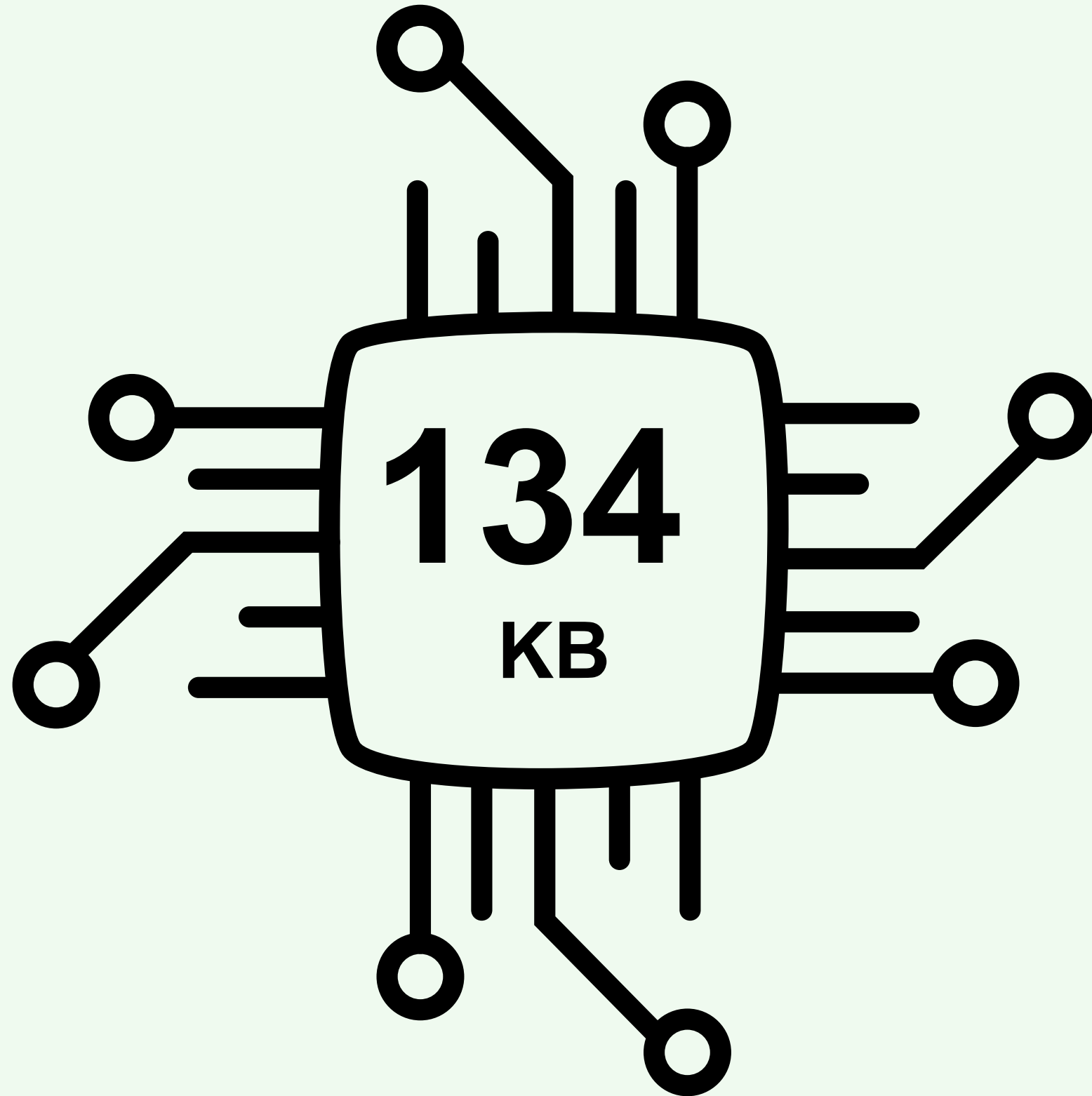
- Full ``int8`` quantization failed.
- The TFLite converter does not natively support the ``tf.ExtractImagePatches`` operation used in the ``Patches`` layer.

Solution: **Mixed Precision**



- I enabled "Flex Ops" (``SELECT_TF_OPS``). This creates a "mixed precision" model

An Efficient, Portable Model

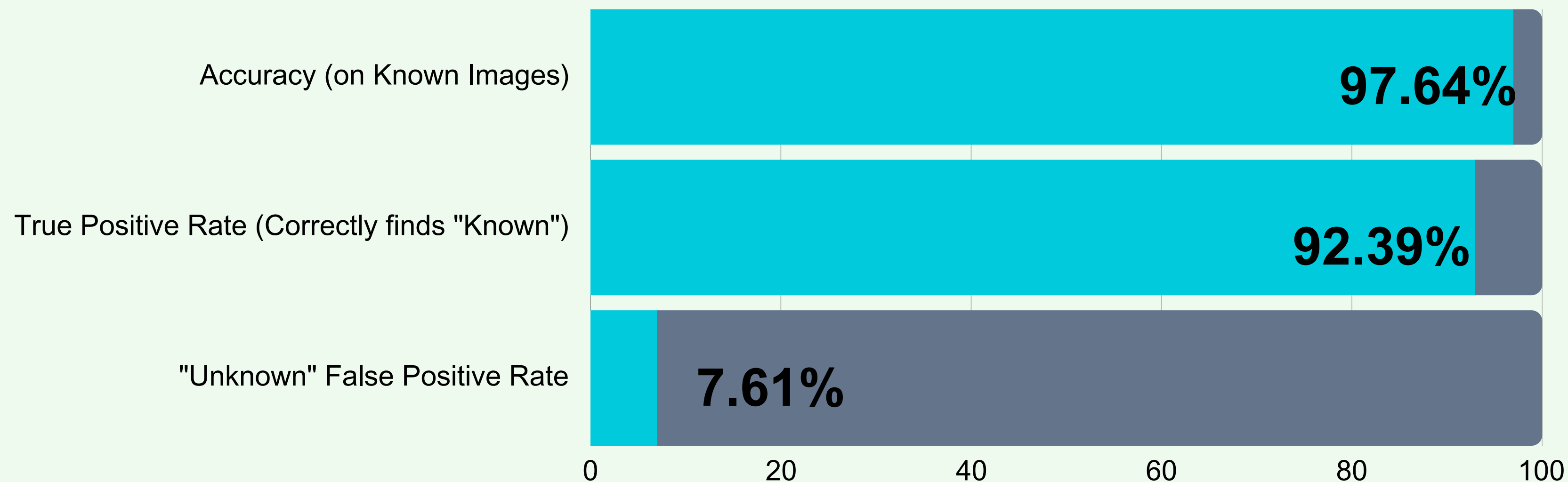


- `vit_extractor_mixed.tflite``

The final quantized model is incredibly small, fast, and ready for on-device inference.

The entire inference pipeline (ViT features + k-NN search) runs in just **~18-30ms** proving the "efficient" part of the goal is met.

Final Evaluation: Unseen Test Set (4,864 Images)



- Conclusion: The system is highly accurate (97.64%) and has a well-balanced "unknown" detector. The 99th percentile threshold correctly allows 92.4% of known images while rejecting 7.6% as "unknown."

Case 1: "Similar but Unknown" (Infant vs. People)



```
Prediction:      Unknown Class  
1-NN Distance:   0.0288 (Threshold is 0.0174)  
Relationship:    ['people', 'people', 'people', 'people', 'people']  
(/DATA/conda-envs/conda-env) user@osman-blurry: /Desktop/askhi-askhi
```

Case 2: Truck vs. Van



```
(/DATA/conda_envs/gdown_env) user@server-bkumar: ~/Desktop  
Prediction: Unknown Class  
1-NN Distance: 0.0328 (Threshold is 0.0174)  
Relationship: ['van', 'van', 'van', 'van', 'van']  
(/DATA/conda_envs/gdown_env) user@server-bkumar: ~/Desktop
```

A Truly Novelty System

Feature	Simple k-NN (Pixels)	Standard ViT (Classifier)	My Hybrid ViT-k-NN (Open-Set)
Understands "Meaning"	✗	✓ (Sees features)	✓ (Sees features)
Detects "Unknowns"	✗	✗	✓
Shows "Relationship"	✗ (Finds wrong neighbors)	✗	✓
Efficient (Quantized)	✗ (Huge DB: 17k * 49k)	✓	✓

- Crucially, the core inference of the novelty model is performed identically to the baseline ViT.

Thank You

